

DAY 04 – APIs, Gemini API & AI Chatbot using Python

Name: M. Lalitha Sowjanya

Team: 4

Date: 12-06-26

1. OBJECTIVE:

To understand APIs, AI APIs, Gemini API, API Keys, and create a simple AI chatbot using Python.

2. TASKS ASSIGNED

2.1. What is an API?

- API stands for Application Programming Interface.
- It is a set of rules that allows one software application to communicate with another software application and exchange data or services in a controlled manner.
- An API acts as a bridge between different software systems.
- Instead of directly accessing another application's internal code or database, a program can send requests through an API and receive the required information in response.

Example:

A weather application provides an API that allows developers to retrieve weather information such as temperature, humidity, and forecasts without accessing the weather provider's internal systems.

2.2. What is an AI API?

- An AI API is an Application Programming Interface that provides access to Artificial Intelligence models and services.
- It allows developers to use AI services such as text generation, summarization, translation, image generation, and question answering within their applications.
- Instead of building and training their own AI models, developers can use AI APIs to access pre-trained AI models and receive AI generated responses.

Example:

Gemini API, OpenAI API, and Claude API

2.3. What is Gemini API?

- Gemini API is an AI API provided by Google that allows developers to access and use the Gemini AI model in their applications.
- Developers can send prompts to the Gemini model and receive AI generated responses.
- This helps developers avoid training and building an AI model themselves.

Advantages:

- Easy integration with applications
- Access to AI services without high cost
- No need to train an AI model

2.4. What is an API Key?

- An API Key is a unique identifier used to authenticate and authorize an application when accessing an API.
- The API key acts as an identity for the application and helps the API provider verify who is making the request.

Importance of API Keys:

- Authentication
- Authorization
- Tracks usage
- Security

2.5. Difference Between a Normal Python Program and an AI API Chatbot

1. Normal Python Program:

- Uses predefined logic written by the programmer.
- Produces fixed and predictable outputs.
- Does not require an API key.
- Usually works offline.

2. AI API Chatbot

- Uses an AI model through an API.
 - Generates dynamic responses.
 - Requires an API key.
 - Requires an internet connection.
-

2.6. Generating Gemini API Key

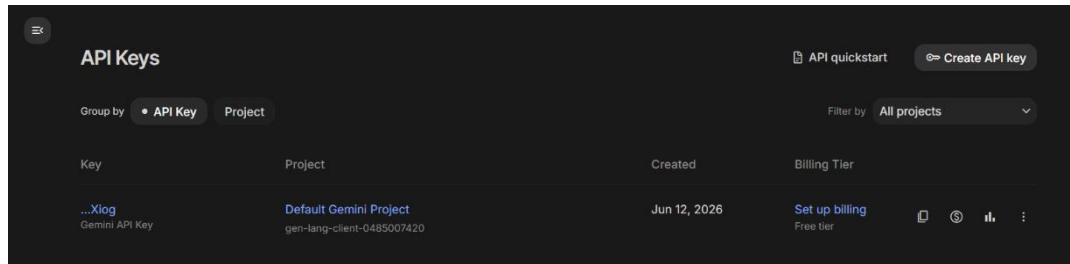


Fig. 1: Gemini API Key in Google AI Studio

2.7. Installation of google-generativeai package

```
# Installing google-generativeai package

!pip install google-generativeai

Requirement already satisfied: google-generativeai in /usr/local/lib/python3.12/dist-packages (0.8.6)
Requirement already satisfied: google-ai-generativelanguage==0.6.15 in /usr/local/lib/python3.12/dist-packages (from google-generativeai) (0.6.15)
Requirement already satisfied: google-api-core in /usr/local/lib/python3.12/dist-packages (from google-generativeai) (2.30.3)
Requirement already satisfied: google-api-python-client in /usr/local/lib/python3.12/dist-packages (from google-generativeai) (2.197.0)
Requirement already satisfied: google-auth==2.15.0 in /usr/local/lib/python3.12/dist-packages (from google-generativeai) (2.47.0)
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packages (from google-generativeai) (5.29.6)
Requirement already satisfied: pydantic in /usr/local/lib/python3.12/dist-packages (from google-generativeai) (2.12.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from google-generativeai) (4.67.3)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.12/dist-packages (from google-generativeai) (4.15.0)
Requirement already satisfied: proto-plus<2.0.0dev,>=1.22.3 in /usr/local/lib/python3.12/dist-packages (from google-ai-generativelanguage==0.6.15->google-generativeai) (1.28.0)
Requirement already satisfied: googleapis-common-protos<2.0.0,>=1.61.2 in /usr/local/lib/python3.12/dist-packages (from google-api-core->google-generativeai) (1.75.0)
Requirement already satisfied: requests<3.0.0,>=2.20.0 in /usr/local/lib/python3.12/dist-packages (from google-api-core->google-generativeai) (2.32.4)
Requirement already satisfied: pyasn1-modules<0.2.1 in /usr/local/lib/python3.12/dist-packages (from google-auth==2.15.0->google-generativeai) (0.4.2)
Requirement already satisfied: rsa<5,>=3.1.4 in /usr/local/lib/python3.12/dist-packages (from google-auth==2.15.0->google-generativeai) (4.9.1)
Requirement already satisfied: httpbin<1.0.0,>=0.19.0 in /usr/local/lib/python3.12/dist-packages (from google-api-python-client->google-generativeai) (0.31.2)
Requirement already satisfied: google-auth-httpbin<1.0.0,>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from google-api-python-client->google-generativeai) (0.4.0)
Requirement already satisfied: uritemplate<5,>=3.0.1 in /usr/local/lib/python3.12/dist-packages (from google-api-python-client->google-generativeai) (4.2.0)
Requirement already satisfied: annotated-types<0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic->google-generativeai) (0.7.0)
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages (from pydantic->google-generativeai) (2.41.4)
Requirement already satisfied: typing-inspection<0.4.2 in /usr/local/lib/python3.12/dist-packages (from pydantic->google-generativeai) (0.4.2)
Requirement already satisfied: grpcio<2.0.0,>=1.33.2 in /usr/local/lib/python3.12/dist-packages (from google-api-core[grpc]>=2.0.*,<=2.1.*,<=2.10.*,<=2.2.*,<=2.3.*,<=2.4.*,<=2.5.*,<=2.6.*,<=2.7.*,<=2.8.*,<=2.9.* in /usr/local/lib/python3.12/dist-packages (from google-api-core[grpc]>=2.0.*,<=2.1.*,<=2.10.*,<=2.2.*,<=2.3.*,<=2.4.*,<=2.5.*,<=2.6.*,<=2.7.*,<=2.8.*,<=2.9.*->google-generativeai) (3.3.2)
Requirement already satisfied: pyasn1<0.7.0,>=0.6.1 in /usr/local/lib/python3.12/dist-packages (from pyasn1-modules==0.2.1->google-auth==2.15.0->google-generativeai) (0.6.3)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.20.0->google-api-core->google-generativeai) (3.4.7)
Requirement already satisfied: idna<3,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.20.0->google-api-core->google-generativeai) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.20.0->google-api-core->google-generativeai) (2.5.0)
Requirement already satisfied: certifi<=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.20.0->google-api-core->google-generativeai) (2026.5.20)
```

Fig. 2: Installation of the google-generativeai Package

```
# Installation checkup

import google.generativeai as genai

print("Package installed successfully!")

Package installed successfully!
/usr/local/lib/python3.12/dist-packages/google/colab/_import_hooks/_hook_injector.py:55: FutureWarning:
All support for the `google.generativeai` package has ended. It will no longer be receiving
updates or bug fixes. Please switch to the `google.genai` package as soon as possible.
See README for more details:
https://github.com/google-gemini/deprecated-generative-ai-python/blob/main/README.md

loader.exec_module(module)
```

Fig. 3: Installation Verification

2.8. Implementation of AI Chatbot using Python

Source Code:

```
import google.generativeai as genai

genai.configure(api_key="API KEY")
```

```

model = genai.GenerativeModel("gemini-2.5-flash")

while True:
    user_input = input("You: ")

    if user_input.lower() == "end":
        print("Chat ended.")
        break

    response = model.generate_content(user_input)
    print("Bot:", response.text)

```

Code Explanation:

1. **import google.generativeai as genai**
 - Imports the Google Generative AI library into the program.
 - This library allows us to access and use Gemini AI.
2. **genai.configure(api_key="API_KEY")**
 - Configures the Gemini API using the API key.
 - The API key is used to authenticate the application and access the Gemini service.
3. **model = genai.GenerativeModel("gemini-2.5-flash")**
 - Creates an instance of the Gemini AI model.
 - The model will be used to generate responses for user prompts.
4. **while True:**
 - Creates an infinite loop.
 - This allows the chatbot to continue running until the user decides to stop it.
5. **user_input = input("You: ")**
 - Accepts a question or message from the user.
 - Stores the input in the variable user_input.
6. **if user_input.lower() == "end":**
 - Checks whether the user entered the word "end".
 - The lower() method converts the input to lowercase for easy comparison.
7. **print("Chat ended.")**
 - Displays a message indicating that the chatbot session has ended.
8. **Break**
 - Terminates the loop and stops the chatbot.

9. `response = model.generate_content(user_input)`

- Sends the user's input to the Gemini AI model.
- The model processes the input and generates a response.
- The generated response is stored in the variable `response`.

10. `print("Bot:", response.text)`

- Displays the response generated by Gemini AI.
- `response.text` contains the actual text returned by the model.

Output:

```
*** /usr/local/lib/python3.12/dist-packages/google/colab/_import_hooks/_hook_injector.py:55: FutureWarning:
All support for the `google.generativeai` package has ended. It will no longer be receiving
updates or bug fixes. Please switch to the `google.genai` package as soon as possible.
See README for more details:
https://github.com/google-gemini/deprecated-generative-ai-python/blob/main/README.md

  loader.exec_module(module)
You: What is Python in a single line?
Bot: Python is a high-level, interpreted, general-purpose programming language known for its clear syntax and versatility across many domains.
You: 1 main feature of python?
Bot: Its highly readable and simple syntax, often enforced by its significant whitespace (indentation), which makes it easy to learn, write, and understand.
You: end
Chat ended.
```

3. LEARNINGS

- Learned how APIs enable communication between applications.
- Understood the concept of AI APIs and Gemini API.
- Learned the importance of API keys for authentication.
- Installed and verified the Google Generative AI package.
- Developed a simple AI chatbot using Python and Gemini API.
- Implemented loops and conditional statements to control chatbot execution.